

Experiment-1

Environment and Connectivity - install required packages(e.g, Transformers, OpenAI, HuggingFace) securely configure the API key and run a simple "Hello World" prompt to verify model access

Program:

Step - 1: Install the Hugging Face Hub Library

In

```
!pip install -U huggingface_hub
```

[49]:

```
Requirement already satisfied: huggingface_hub in c:\users\neeha\anaconda3\lib\site-packages (1.22.0)
Requirement already satisfied: click<9.0.0,>=8.4.2 in c:\users\neeha\anaconda3\lib\site-packages (from huggingface_hub) (8.4.2)
Requirement already satisfied: filelock>=3.10.0 in c:\users\neeha\anaconda3\lib\site-packages (from huggingface_hub) (3.13.1)
Requirement already satisfied: fsspec>=2023.5.0 in c:\users\neeha\anaconda3\lib\site-packages (from huggingface_hub) (2024.3.1)
Requirement already satisfied: hf-xet<2.0.0,>=1.5.1 in c:\users\neeha\anaconda3\lib\site-packages (from huggingface_hub) (1.5.1)
Requirement already satisfied: httpx<1,>=0.23.0 in c:\users\neeha\anaconda3\lib\site-packages (from huggingface_hub) (0.26.0)
Requirement already satisfied: packaging>=20.9 in c:\users\neeha\anaconda3\lib\site-packages (from huggingface_hub) (23.2)
Requirement already satisfied: pyyaml>=5.1 in c:\users\neeha\anaconda3\lib\site-packages (from huggingface_hub) (6.0.1)
Requirement already satisfied: tqdm>=4.42.1 in c:\users\neeha\anaconda3\lib\site-packages (from huggingface_hub) (4.66.4)
Requirement already satisfied: typing-extensions>=4.1.0 in c:\users\neeha\anaconda3\lib\site-packages (from huggingface_hub) (4.16.0)
```

Step - 2: Load the Hugging Face API Token from the Environment File

In [19]:

```
from huggingface_hub import InferenceClient
import os
from dotenv import load_dotenv
```

```
else:  
    print("ERROR!! NO API KEY")  
API key exists  
API key: hf_ANL...RtAx
```

Step - 3: Load The Model

```
In  
[51]: client = InferenceClient(api_key=api_key)  
       response=client.chat.completions.create(  
           model="Qwen/Qwen2.5-7B-Instruct",  
           messages=[  
               {  
                   "role": "user",  
                   "content": "Hello, World"  
               }  
           ],  
           max_tokens=50  
       )
```

Hello, World! It's nice to meet you. Is there anything specific you'd like to chat about or any questions you have?

Step – 4 : Generate a Response Using the Hugging Face Inference API

```
response = client.chat.completions.create(  
    model="Qwen/Qwen2.5-7B-Instruct",  
    messages=[  
        {  
            "role": "user",  
            "content": "explain AI"  
        }  
    ],  
    max_tokens=200  
)  
print(response.choices[0].message.content)
```

Artificial Intelligence (AI) is a broad field of study that focuses on creating machines capable of performing tasks that typically require human intelligence. At its core, AI involves developing algorithms and models that can learn from and make decisions based on data. Machine learning, a subset of AI, enables computers to learn from and improve through experience without being explicitly programmed. Deep learning, a more advanced form of machine learning, uses neural networks with multiple layers to recognize patterns in complex data, such as images, sound, and text. Natural language processing (NLP) is a branch of AI that focuses on the interaction between humans and computers using natural language. AI applications range from simple tasks like recommending products on an e-commerce site to complex systems like autonomous vehicles and medical diagnosis tools. AI has the potential to revolutionize industries by au

tomating processes, improving efficiency, and enhancing decision-making. However, it also raises ethical concerns, such as privacy, bias, and the impact on employment. As AI technology continues to advance

Experiment - 2: Baseline vs. Enhanced Prompts: Execute a naïve prompt (“Write a one-paragraph bio of Ada Lovelace.”) and an enhanced prompt that adds role framing, specificity, and explicit format instructions; compare both outputs for relevance, completeness, and style.

Step – 1: Import the Required Library

```
from transformers import pipeline
```

Step – 2: Load the Text Generation Model

```
generator = pipeline(  
    "text-generation",  
    model="distilgpt2"  
)
```

Step – 3: Define the Naive Prompt

```
naive_prompt = "Write a one-  
paragraph bio of Ada Lovelace."
```

Step – 4: Define the Enhanced Prompt

```
enhanced_prompt = """  
    You are a technology historian writing for a university newsletter.  
    Write a biography of Ada Lovelace.  
    Requirements:  
    1. Length: 120-150 words  
    2. Include:  
    - Birth and death years  
    - Collaboration with Charles Babbage  
    - Contribution to the Analytical Engine  
    - Why she is considered the first computer programmer  
    3. Tone: Professional and engaging  
    4. Format: Exactly one paragraph """
```

Step – 5: Create a Function to Generate Responses

```
def get_response(prompt):  
    output = generator(  
        prompt,      max_length=200,  
        num_return_sequences=1,  
        truncation=True  
    )  
    return output[0]["generated_text"]
```

Step – 6: Generate Output for the Naive Prompt and Enhanced Prompt

```
naive_output = get_response(naive_prompt)  
print("=="*80)
```

```
print("NAIVE PROMPT OUTPUT")
print("="*80)
print(naive_output)
enhanced_output = get_response(enhanced_prompt)
print("="*80)
print("ENHANCED PROMPT OUTPUT")
print("="*80)
print(enhanced_output)
```

Step – 7: Compare the Outputs

```
print("="*80)
print("COMPARISON")
print("="*80)
print("""
```

1. Relevance:
 - Enhanced prompt is more focused.
 - Naive prompt is less specific.
2. Completeness:
 - Enhanced prompt includes all required details.
 - Naive prompt may miss important facts.

- ### 3. Style:
- Enhanced prompt produces a structured professional biography.
 - Naive prompt depends entirely on the model.

Conclusion:

Enhanced prompts generate more accurate, relevant, and well-structured responses. """)

OUTPUT:

```
PS C:\Users\grace\Downloads\Pragmatics> python hf22.py
Loading weights: 100% |#####| 76/76 [00:00:00.00, 7208.83it/s]
Warning: You are sending unauthorized requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
[transformers] Packing 'generation_config' together with generation-related arguments ({'max_length': 'max_return_sequences'}) is deprecated and will be removed in a future version. Please pass either a 'generation_config' object OR all generation parameters explicitly, but not both.
[transformers] Both 'max_new_tokens' (~250) and 'max_length' (~200) seem to have been set. 'max_new_tokens' will take precedence. Please refer to the documentation on how to use them. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
[transformers] Ignoring clean_up_tokenization_spaces=True for BPE tokenizer GPT2Tokenizer. The clean_up_tokenization_post-processing step is designed for WordPiece tokenizers and is destructive for BPE (it strips spaces before punctuation).
Set clean_up_tokenization_spaces=False to suppress this warning, or set clean_up_tokenization_spaces_for_bpe even though it will interrupt output=True to force cleaning anyway.
[transformers] Both 'max_new_tokens' (~250) and 'max_length' (~200) seem to have been set. 'max_new_tokens' will take precedence. Please refer to the documentation on how to use them. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
```

NATIVE PROMPT OUTPUT

Write a one-paragraph bio of Ada Lovelace.

An interesting piece of paper, "Ada, is a computer programmer. Ada is used for a

```

PS C:\Users\prasa\Downloads\PromptLab> python hf22.py
You are a technology historian writing for a university newsletter.
Write a biography of Ada Lovelace.
Requirements:
1. Length: 120-150 words
2. Include:
  - Birth and death years
  - Collaboration with Charles Babbage
  - Contribution to the Analytical Engine
  - Why she is considered the first computer programmer
3. Tone: Professional and engaging
4. Format: Exactly one paragraph

5. Length: 180-200 words
6. Format: Exactly one paragraph
7. Format: Exactly one paragraph
8. Format: Exactly one paragraph
9. Format: Exactly one paragraph
10. Format: Exactly one paragraph
11. Format: Exactly one paragraph
12. Format: Exactly one paragraph
13. Format: Exactly one paragraph
14. Format: Exactly one paragraph
15. Format: Exactly one paragraph
16. Format: Exactly one paragraph
17. Format: Exactly one paragraph
18. Format: Exactly one paragraph
19. Format: Exactly one paragraph
20. Format: Exactly one paragraph
21. Format: Exactly one paragraph
22. Format: Exactly one paragraph
23. Format: Exactly one paragraph
24. Format: Exactly one paragraph
25. Format: Exactly one paragraph
26. Format: Exactly one paragraph
27. Format: Exactly one paragraph
28. Format: Exactly one paragraph
29. Format: Exactly one paragraph
30. Format: Exactly one paragraph
31. Format: Yes
32. Format: Yes
33. Format: Yes
34. Format: Yes
35. Format: Yes
36. Format: Yes
37. Format: Yes
38. Format: Yes

=====
COMPARISON
=====

1. Relevance:
  - Enhanced prompt is usually more focused on Ada Lovelace's achievements.
  - Naive prompt may contain less targeted information.

2. Completeness:
  - Enhanced prompt explicitly requests important facts.
  - Naive prompt may omit key details.

3. Style:
  - Enhanced prompt produces a professional and structured biography.
  - Naive prompt style depends entirely on the model.

Conclusion:
The enhanced prompt generally produces more relevant, complete, and well-structured output because it includes role framing, specificity, and formatting instructions.

PS C:\Users\prasa\Downloads\PromptLab>
PS C:\Users\prasa\Downloads\PromptLab>
PS C:\Users\prasa\Downloads\PromptLab> python hf22.py
Certainly! Transfer learning is a powerful technique in machine learning where a model developed for a task is reused as the starting point for a model on a second task. This approach is particularly useful when the second task has a limited amount of labeled data or when the model can benefit from the learned features of a related task.

### Key Concepts of Transfer Learning

```


Experiment 3: Iterative Refinement on a Simple Task: Summarize the plot of the Shakespearean play Romeo and Juliet in two sentences through three rounds of prompt tweaking: a. Minimal instruction. b. Addition of length and style constraints c. Specification of key content elements (setting and theme) Document how each iteration changes and improves the result.

Step – 1: Import the Required Libraries

```
from dotenv import load_dotenv
import os
from huggingface_hub import InferenceClient
```

Step – 2: Load the Hugging Face API Token

```
load_dotenv(".env")
HF_TOKEN = os.getenv("huggingface")
if HF_TOKEN is None:
    raise ValueError("Hugging Face token not found in .env")
```

Step – 3: Create the Hugging Face Inference Client

```
client = InferenceClient(
    api_key=HF_TOKEN
)
```

Step – 4: Create a Function to Generate Responses

```
def get_response(prompt):
    completion = client.chat.completions.create(
        model="Qwen/Qwen2.5-7B-Instruct",
        messages=[
            {
                "role": "user",
                "content": prompt
            }
        ],
        max_tokens=200,
        temperature=0.7,
    )
    return completion.choices[0].message.content.strip()
```

Step – 5: Define Prompt – Iteration 1

```
prompt1 = """
Summarize the plot of the Shakespearean play Romeo and Juliet. """
```

Step – 6: Define Prompt – Iteration 2

```
prompt2 = """
Summarize the plot of the Shakespearean play Romeo and Juliet in exactly two sentences.
Use a clear, formal, and concise writing style in English. """
```

Step – 7: Define Prompt – Iteration 3

```
prompt3 = """
Summarize the plot of the Shakespearean play Romeo and Juliet in exactly two sentences.
Use a clear, formal, and concise style.
Include:
```

1. The setting (Verona, Italy)
2. The conflict between the Montague and Capulet families
3. The tragic ending
4. The central theme of love versus hatred """

Step – 8: Generate Responses

```
output1 = get_response(prompt1)
```

```
output2 = get_response(prompt2) output3
```

```
= get_response(prompt3) Step – 9:
```

```
Display the Responses print("="
```

```
* 80)
```

```
print("ITERATION 1: Minimal Instruction")
```

```
print("=" * 80)
```

```
print(output1) print("\n"
```

```
+ "=" * 80)
```

```
print("ITERATION 2: Added Length and Style
```

```
Constraints") print("=" * 80) print(output2) print("\n" +
```

```
"=" * 80)
```

```
print("ITERATION 3: Added Key Content
```

```
Elements") print("=" * 80) print(output3)
```

Step – 10: Display the Improvement Analysis

```
print("=" * 80) print("DOCUMENTATION OF
```

```
IMPROVEMENTS") print("=" * 80) print("""
```

```
Iteration 1:
```

- Uses only a minimal instruction.
- The response length and style may vary.
- Important details may be omitted.

```
Iteration 2:
```

- Adds an exact length requirement (two sentences).
- Requests a formal and concise writing style.
- Produces a more focused and consistent summary.

```
Iteration 3:
```

- Specifies the essential content to include:
 - Setting (Verona, Italy)
 - Family conflict
 - Tragic ending
 - Theme (Love versus Hatred)
- Produces the most informative and structured summary while satisfying all constraints. """)

OUTPUT:

```
Python 3.9.6
> python 9b3.py
=====
Iteration 1: Minimal Instruction
=====
Romeo and Juliet is a tragic play written by William Shakespeare. The play revolves around two young star-crossed lovers whose deaths ultimately reconcile their feuding families.

Here's a summary of the key events:

1. "Setting and Initial Conflict": The play is set in the city of Verona, Italy, where the noble families, the Montagues and the Capulets, are engaged in a long-standing feud.

2. "Meeting and Love": Romeo, a Montague, and Juliet, a Capulet, meet at a party and fall deeply in love. They decide to marry in secret.

3. "Secret Marriage": Friar Laurence, a kind and well-intentioned priest, agrees to marry them in an attempt to end the longstanding feud between the families by uniting the young lovers.

4. "Tragedy": Romeo and Juliet's marriage is kept secret from their families. A series of unfortunate events, including a miscommunication of letters and a series of coincidences, result in a cascade of tragic events culminating in their deaths.

=====
Iteration 2: Added Length and Style Constraints
=====
Romeo and Juliet is a tragic play by William Shakespeare about the forbidden love between Romeo, a young Montague, and Juliet, a young Capulet, whose families are enmeshed in a long-standing feud, leading to their tragic deaths. Despite his efforts to be together, the couple's secret marriage by Friar Laurence and a series of miscommunications result in a cascade of tragic events culminating in their deaths.

=====
Iteration 3: Added Key Content Elements
=====
Set in Verona, Italy, "Romeo and Juliet" is a tragic play that explores the complex interplay between the Montague and Capulet families. Their longstanding feud fuels a destructive hatred that ultimately leads to the lovers' suicides, highlighting the devastating consequences when love is pitted against family.

=====
DOCUMENTATION OF IMPROVEMENTS
=====
Iteration 1:
Uses only a minimal instruction.
The response length and style may vary.
Important details may be omitted.

Iteration 2:
Adds an exact length requirement (two sentences).
Requests a formal and concise writing style.
Produces a more focused and consistent summary.

Iteration 3:
Specifies the essential content to include:
  • Setting (Verona, Italy)
  • Family conflict
  • Tragic ending
  • Theme (Love versus Hatred)
Produces the most informative and structured summary while satisfying all constraints. """)
```


Experiment 4: Diagnosing Prompt Failures & Edge Cases: Craft a vague or contradictory prompt; analyze the failure mode (ambiguity, missing context, or format errors); refine the prompt by adding examples or clarifying instructions

Step – 1: Import the Required Library

```
from transformers import pipeline
```

Step – 2: Load the Text Generation Model

```
generator = pipeline(
    "text-generation",
    model="TinyLlama/TinyLlama-1.1B-Chat-v1.0"
)
```

Step – 3: Create a Vague Prompt `vague_prompt =`

`"Write about Python."` **Step – 4: Generate Response**

for the Vague Prompt

```
print("=" * 60)
print("VAGUE PROMPT")
print("=" * 60)
print("Prompt:", vague_prompt)
output1 = generator(
    vague_prompt,
    max_new_tokens=80,
    do_sample=False
)
print("\nGenerated Output:\n")
print(output1[0]["generated_text"]) Step – 5:
```

Create a Contradictory Prompt

```
contradictory_prompt = """Write exactly 30 words Explain Machine Learning in detail with five examples."""
```

Step – 6: Generate Response for the Contradictory

Prompt `print("=" * 60) print("CONTRADICTIONARY PROMPT")`

```
print("=" * 60)
print("Prompt:", contradictory_prompt)
output2 = generator(
    contradictory_prompt,
    max_new_tokens=120,
    do_sample=False
)
print("\nGenerated      Output:\n")
print(output2[0]["generated_text"]) Step –
```

7: Create a Refined Prompt

```
refined_prompt = """
```

You are a computer science teacher.

Task:

Write a paragraph (120-150 words) explaining Machine Learning. Requirements:

1. Use simple English.

2. Include five applications:

- Healthcare

- Education

- Banking

- Agriculture

- Transportation
- 3. End with a short conclusion.
- """"

Step – 8: Generate Response for the Refined Prompt

```
print("=" * 60)
print("REFINED PROMPT")
print("=" * 60)
print("Prompt:", refined_prompt)
output3 = generator(
    refined_prompt,
    max_new_tokens=180,
    do_sample=False
)
print("\nGenerated Output:\n")
print(output3[0]["generated_text"])
```

9: Compare the Responses

- ```
print("=" * 60)
print("ANALYSIS")
print("=" * 60)
print("""
1. Vague Prompt:
- Gives only a general instruction.
- The response may lack detail and focus.
2. Contradictory Prompt:
- Contains conflicting requirements.
- The model may not satisfy all instructions.
3. Refined Prompt:
- Clearly specifies the task, length, style, and required content.
- Produces a well-structured, informative, and relevant response. """)
```

### OUTPUT:

The screenshot displays a Jupyter Notebook with three cells. The first cell shows the execution of a script that generates a refined prompt. The second cell shows the execution of a script that compares the responses. The third cell shows the execution of a script that generates a response for the refined prompt. The output of the third cell is a well-structured response about Python, including a task, requirements, and a conclusion.

```
Python Prompt
=====
Prompt: Write about Python.

[Transformers] Packing generation_config together with generation-related args
into {'do_sample': True, 'max_tokens': 1} is deprecated and will be removed in 4.45
and removed in 4.46. Please pass them as 'generation_config' object in all generation
parameters explicitly, but not both.

[Transformers] Both 'max_new_tokens' (-180) and 'max_length' (-1800) seem to have
been set. 'max_new_tokens' will take precedence. Please refer to the documentati
on for more information. (https://huggingface.co/docs/transformers/main/en/whats
_new_classes/transformers)

[Transformers] Spawning class as tokenization space-free for gpt tokenizer-like
attention. The class as tokenization post-processing step is required for some
late vocabulary and is instructive for the (it defines space before padding).
Set class as tokenization space-free to suppress this warning, or set class
as tokenization space-free as you want, though it will corrupt output-text to force
cleaner anyway.

Generated Output:
=====
Write about Python.

=====
CONTRADICTION PROMPT
=====
Prompt:
=====
Write exactly 30 words.
Explain machine learning in detail with five examples.
Include at least one statistic for each related to the topic.
Use proper formatting (e.g., headings, subheadings, bullet points).
Include a bibliography or reference list.

[Transformers] Both 'max_new_tokens' (-180) and 'max_length' (-1800) seem to have
been set. 'max_new_tokens' will take precedence. Please refer to the documentati
on for more information. (https://huggingface.co/docs/transformers/main/en/whats
_new_classes/transformers)

[Transformers] Spawning class as tokenization space-free for gpt tokenizer-like
attention. The class as tokenization post-processing step is required for some
late vocabulary and is instructive for the (it defines space before padding).
Set class as tokenization space-free to suppress this warning, or set class
as tokenization space-free as you want, though it will corrupt output-text to force
cleaner anyway.

Generated Output:
=====
Write about Python.

=====
REFINED PROMPT
=====
Prompt:
=====
You are a computer science teacher.
Task:
=====
Write a paragraph (120-150 words) explaining machine learning.
Requirements:
=====
1. Use simple English.
2. Include five applications:
- Healthcare
- Education
- Banking
- Agriculture
- Transportation
3. End with a short conclusion.

In healthcare, machine learning is used to predict patient outcomes, such as the
likelihood of a patient developing a certain disease or condition. This can hel
p doctors make more informed decisions about treatment and prevention. For exam
ple, a machine learning algorithm can analyze patient data and predict which pati
ents are at high risk of developing a certain disease. This information can then
be used to develop personalized treatment plans for each patient.

In education, machine learning is used to improve learning outcomes. For example
, a machine learning algorithm can analyze student data and identify which stud
ents are
Python Prompt
=====
Prompt: Write about Python.

[Transformers] Packing generation_config together with generation-related args
into {'do_sample': True, 'max_tokens': 1} is deprecated and will be removed in 4.45
and removed in 4.46. Please pass them as 'generation_config' object in all generation
parameters explicitly, but not both.

[Transformers] Both 'max_new_tokens' (-180) and 'max_length' (-1800) seem to have
been set. 'max_new_tokens' will take precedence. Please refer to the documentati
on for more information. (https://huggingface.co/docs/transformers/main/en/whats
_new_classes/transformers)

[Transformers] Spawning class as tokenization space-free for gpt tokenizer-like
attention. The class as tokenization post-processing step is required for some
late vocabulary and is instructive for the (it defines space before padding).
Set class as tokenization space-free to suppress this warning, or set class
as tokenization space-free as you want, though it will corrupt output-text to force
cleaner anyway.

Generated Output:
=====
Write about Python.

=====
CONTRADICTION PROMPT
=====
Prompt:
=====
Write exactly 30 words.
Explain machine learning in detail with five examples.
Include at least one statistic for each related to the topic.
Use proper formatting (e.g., headings, subheadings, bullet points).
Include a bibliography or reference list.

[Transformers] Both 'max_new_tokens' (-180) and 'max_length' (-1800) seem to have
been set. 'max_new_tokens' will take precedence. Please refer to the documentati
on for more information. (https://huggingface.co/docs/transformers/main/en/whats
_new_classes/transformers)

[Transformers] Spawning class as tokenization space-free for gpt tokenizer-like
attention. The class as tokenization post-processing step is required for some
late vocabulary and is instructive for the (it defines space before padding).
Set class as tokenization space-free to suppress this warning, or set class
as tokenization space-free as you want, though it will corrupt output-text to force
cleaner anyway.

Generated Output:
=====
Write about Python.

=====
REFINED PROMPT
=====
Prompt:
=====
You are a computer science teacher.
Task:
=====
Write a paragraph (120-150 words) explaining machine learning.
Requirements:
=====
1. Use simple English.
2. Include five applications:
- Healthcare
- Education
- Banking
- Agriculture
- Transportation
3. End with a short conclusion.

In healthcare, machine learning is used to predict patient outcomes, such as the
likelihood of a patient developing a certain disease or condition. This can hel
p doctors make more informed decisions about treatment and prevention. For exam
ple, a machine learning algorithm can analyze patient data and predict which pati
ents are at high risk of developing a certain disease. This information can then
be used to develop personalized treatment plans for each patient.

In education, machine learning is used to improve learning outcomes. For example
, a machine learning algorithm can analyze student data and identify which stud
ents are
```